FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF CSE – AI, ML AND DS

PROJECT REPORT

The Butterfly Effect

AI-Assisted Web Application Project

Semester 3 – CSE (AI/ML)

Student details	
Student name	Raj A. Khakhkhar
GR number / Roll number	92610122002
Division	3EI1
College / institute	Marwadi University
University	Marwadi University
Faculty / guide	Chandreshkumar Vyas
Academic year / semester	2026 / Semester 3 – CSE (AI/ML)

Document prepared 29 October 2026

Contents

• 1. Introduction

• 2. Target User

• 3. Tools Used

• 4. Prompt Engineering Process

• 5. Application Design and Development

• 6. Final Application

• 7. Conclusion

1. Introduction

Planning an event involves connected decisions. Increasing the guest count changes catering expenditure; expenditure affects the remaining budget; and reducing preparation time raises modeled pressure and schedule risk. Considering each choice separately can hide these links.

The Butterfly Effect is a browser-based event-planning scenario demonstrator. It lets users change six planning inputs and immediately see calculated spending, budget balance, preparation pressure, schedule risk, and readiness. The dependency map highlights pathways from changed inputs to modeled outcomes, and a fixed baseline remains available for comparison.

The application uses deterministic TypeScript rules. AI tools supported repair, refinement, and documentation of the existing application; the application itself does not call an AI service at runtime. The model illustrates simplified relationships and does not forecast real events or establish scientific causality.

2. Target User

The intended users are university students, instructors, and demonstration audiences who want to explore how a small set of event-planning assumptions changes a simplified model. The interface is designed for scenario exploration and discussion, not professional event budgeting. No user study or survey was conducted, so the project makes no claims about measured learning outcomes or user satisfaction.

3. Tools Used

Technology versions follow the project manifest and lockfile. The application has no backend, paid API, or runtime AI dependency. The exact local editor or IDE used for all earlier coding is not established by the supplied record.

4. Prompt Engineering Process

The process was organized as six stages: define the problem, specify requirements, guide interface design, direct code work, verify behavior, and refine demonstrated issues. Three actual Codex user prompts are reproduced in the separate prompt book. The initial OpenCode prompt history could not be recovered. No image files showing actual prompt conversations were supplied, so application screenshots are not used as substitutes.

4.1 Idea and problem analysis

Purpose: Define the event-planning problem and why linked outcomes are hard to understand from isolated figures.

Available evidence: Prompt 1 asks for inspection of the existing project and diagnosis of actual issues. The earlier OpenCode prompt is unavailable.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

4.2 Requirements and features

Purpose: Specify inputs, outputs, presets, baseline comparison, reset, and evidence standards before implementation.

Available evidence: The available prompts enumerate controls, simulation outcomes, graph, baseline comparison, presets, reset, and evidence rules.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

4.3 UI and layout design

Purpose: Describe visual hierarchy and interaction style while retaining clarity and usability.

Available evidence: Prompt 2 requests a polished responsive dark interface, clear separation of inputs/results/relationships, and restrained transitions. A separate historical UI prompt was not supplied.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

4.4 Code generation and implementation

Purpose: Direct changes to the existing codebase while preserving useful work and architecture.

Available evidence: Prompts 1 and 2 request targeted changes to the existing React/TypeScript application rather than a replacement.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

4.5 Testing and verification

Purpose: Check calculations and real interactions; distinguish completed checks from gaps.

Available evidence: Prompts 1 and 2 request build, lint, tests, and browser checks. Records report successful build/lint, six tests, preset/reset interactions, and a local preview.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

4.6 Debugging and improvement

Purpose: Use observed issues to refine behavior, rerun checks, and record remaining gaps.

Available evidence: Recorded refinements include input validation, zero-budget handling, illustrative-model wording, local font fallbacks, and six Node tests. Original debugging follow-up prompts are unavailable.

[STUDENT ACTION REQUIRED: Insert an authentic screenshot of the relevant prompt conversation for this stage if required. Do not use an application screenshot here.]

Prompt-image evidence status: zero genuine prompt screenshots were included in the supplied materials. The prompt book reproduces three available prompt texts, not screenshots. No OpenCode screenshots can be supplied from the available records.

5. Application Design and Development

The interface is implemented in src/App.tsx. It owns the current scenario and selected graph node, renders controls and presets, calls simulation functions, and displays current results and comparisons. The calculation layer in src/simulation.ts contains scenario types, ranges, validation, baseline values, and deterministic calculations. tests/simulation.test.mjs contains automated tests. Vite serves development and produces static files under dist.

The page presents six inputs: venue cost, guest count, catering cost per guest, travel cost, available budget, and days remaining. Catering expenditure equals guest count multiplied by cost per guest. Total expenditure adds venue, catering, and travel costs. Remaining budget is budget minus spending; deficit is positive overspend or zero. Preparation pressure uses days remaining on a bounded 0–100 scale; schedule risk is derived from pressure; readiness uses budget balance, risk, and time remaining.

Workflow: the user edits a slider or numeric value; the input is bounded to its allowed range and step; React updates scenario state; the simulation recalculates; and result cards, dependency paths, and baseline comparisons refresh. Selecting a graph node highlights its modeled downstream path. Reset copies the fixed baseline into current scenario state.

Page and system structure

Figure 1. Application dashboard showing the scenario controls and calculated results.

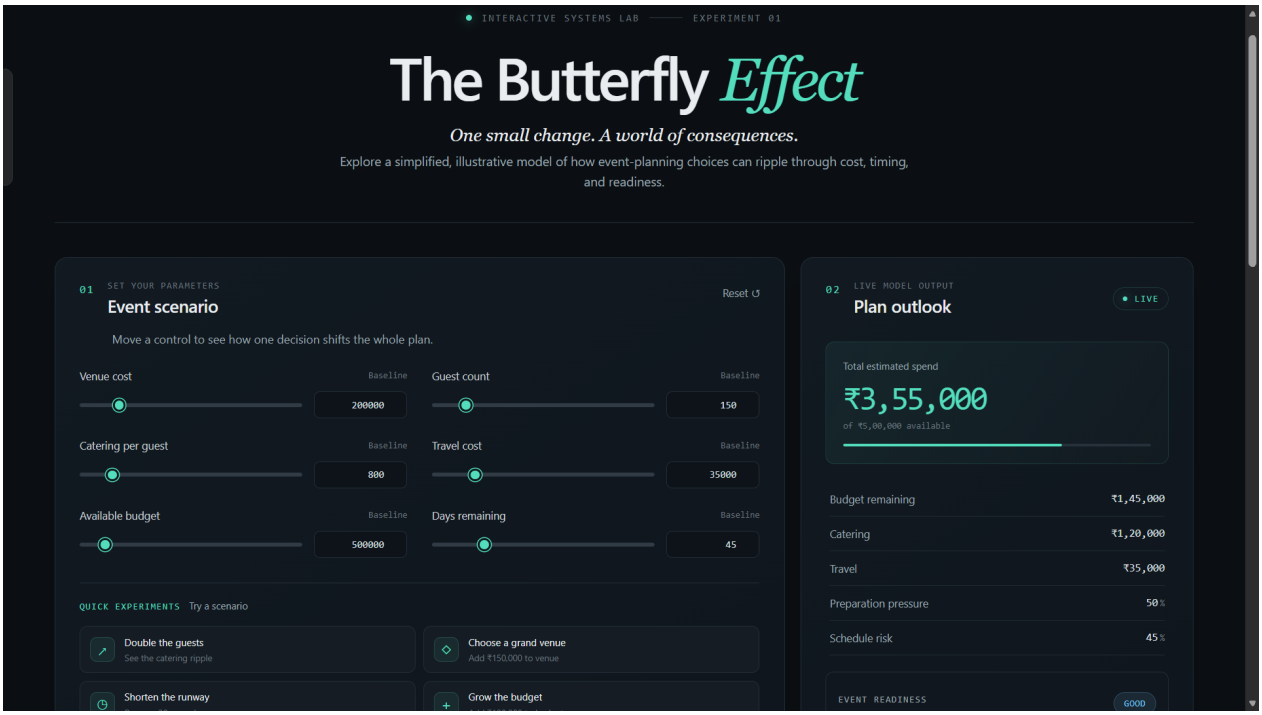


Figure 2. Ripple map showing modeled links between scenario inputs and outcomes.

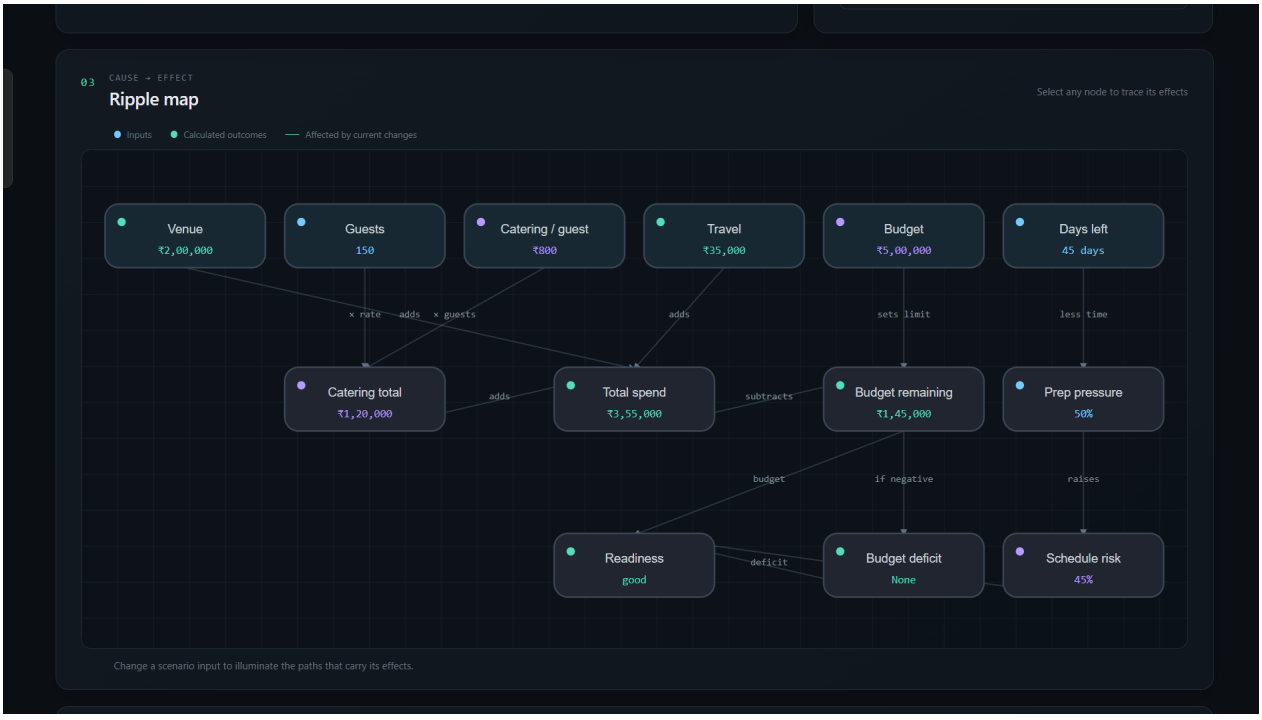
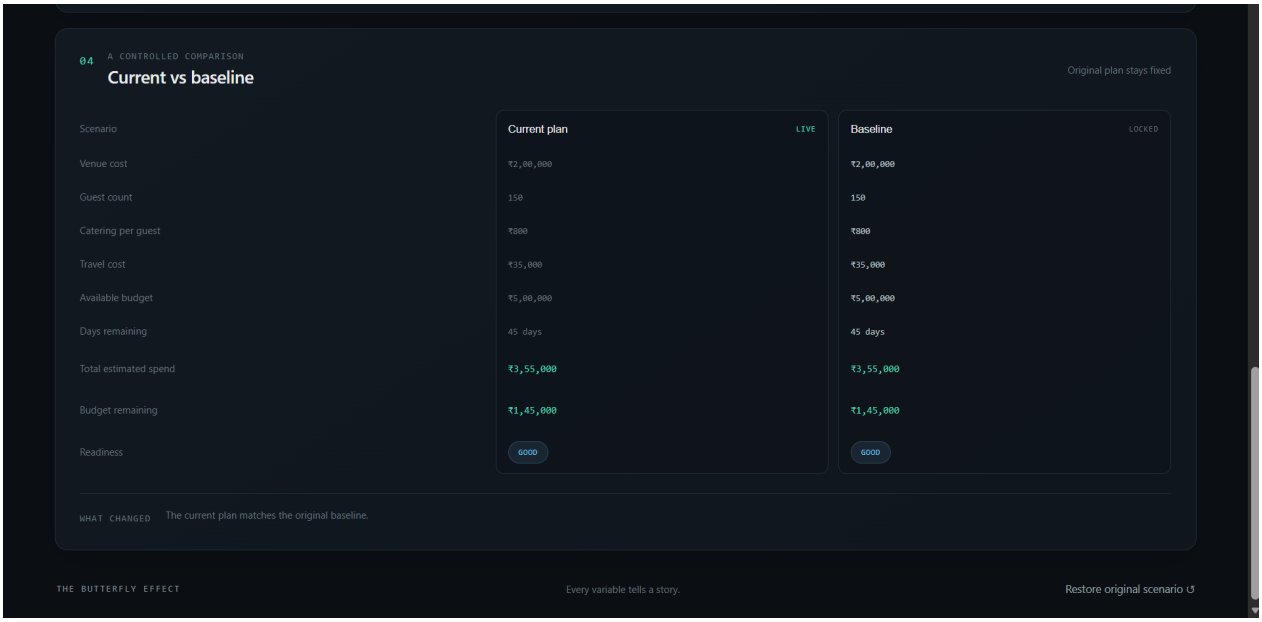


Figure 3. Baseline comparison showing current values against the fixed starting scenario.



Testing record

6. Final Application

The finished local application provides six editable scenario controls; deterministic cost, budget, pressure, risk, and readiness results; an interactive SVG dependency map; current-versus-baseline comparison; four presets; and reset to the default scenario. It is a frontend-only Vite application and needs no server-side service or API key.

A local production preview was observed at http://127.0.0.1:4173/ during prior verification. The development server address is http://127.0.0.1:5173/ while running. Deployment has not been completed, so there is no verified public application URL.

To run locally in PowerShell: cd D:\butterfly-effect, then npm run dev. To prepare a production build: npm run build. Add a public link only after deployment and browser verification of that URL and key interactions.

7. Conclusion

The Butterfly Effect demonstrates how changing event-planning assumptions can affect related outcomes in a transparent deterministic model. It combines a React interface, TypeScript calculations, an SVG dependency map, a fixed baseline, presets, and reset. AI-assisted work supported repair, implementation refinement, testing, and documentation; the available records include successful build and lint runs, six simulation tests, and browser interaction checks.

Limitations include simplified illustrative rules, no user research, incomplete browser-console and graph-keyboard evidence, unavailable original OpenCode prompts and prompt screenshots, and no public deployment. Future work could document authentic prompt evidence, complete those interaction checks, deploy and verify a public build if required, and expand the model only where reliable event-planning data supports it.

References

Project source files: src/App.tsx, src/simulation.ts, src/index.css, tests/simulation.test.mjs, package.json, and package-lock.json. Checked during preparation of this report.

The Butterfly Effect project handover: PROJECT_HANDOVER.md. Used to establish scope and prior verification records, checked against source files and recorded outputs.

University report-format cover image supplied by the student: IPE_Project_report_Format.docx.png. The image shows cover information only; complete section and formatting rules were not available.

Local project access

From PowerShell, run cd D:\butterfly-effect followed by npm run dev. The application is available at http://127.0.0.1:5173/ while the development server is running.